GitHub Pages to Backend Connectivity Analysis & Zero-Touch Architecture Plan

Eliminating Client DNS Changes & Manual Tunnel Entry for the Nueva Ecija Travel Itinerary Application

1. Root-Cause Diagnosis of the Current Connection Architecture

An inspection of the uploaded repository (`scripts/start_server.ps1`, `Project/src/environments/environment.prod.ts`, `Project/src/app/core/services/connectivity.service.ts`, and `Project/src/app/core/interceptors/auth.interceptor.ts`) reveals how the static Angular frontend on GitHub Pages (https://edselrenz.github.io/Itinerary-App/) currently communicates with the local FastAPI backend (`127.0.0.1:8000`) and why users encounter two major connectivity barriers.

1.1 How the Current Dual-Tunnel Mechanism Works in Code

- **Local Dual-Tunnel Launcher (`scripts/start_server.ps1`, Lines 86–173):** When the developer runs `start_server.bat`, PowerShell starts FastAPI on port `8000` alongside two tunnel processes:
  1. **Fixed Free ngrok Domain (Lines 93–119):** Launches `ngrok.exe http --url=https://anybody-cube-surprise.ngrok-free.dev 8000`.
  2. **Ephemeral Cloudflare Quick Tunnel (Lines 122–152):** Launches `cloudflared.exe tunnel --url http://127.0.0.1:8000 --no-autoupdate` and scrapes the randomly generated `https://<random-words>.trycloudflare.com` URL from stderr logs.
- **Frontend Endpoint Resolution (`Project/src/environments/environment.prod.ts`, Lines 21–48):** Every time an Angular service accesses `environment.apiBaseUrl`, `resolveApiBaseUrl()` checks `window.localStorage.getItem('ne_custom_api_url')` against `isTrustedApiHostname()` (`*.ngrok-free.dev`, `*.trycloudflare.com`, `localhost`). If no custom URL is saved in `localStorage`, it defaults to `https://anybody-cube-surprise.ngrok-free.dev/api/v1`.
- **Interceptor Header Injection (`Project/src/app/core/interceptors/auth.interceptor.ts`, Lines 19–30):** Attaches `'ngrok-skip-browser-warning': 'true'` and `Authorization: Bearer <token>` to requests matching `environment.apiBaseUrl`, and triggers `connectivityService.setHostOffline(true)` whenever an HTTP call fails with status `0` (DNS/CORS/network failure), `502`, or `504`.

1.2 Why Problem #1 Happens: Users Must Change DNS to Cloudflare (1.1.1.1) or Google (8.8.8.8)

Philippine residential ISPs (PLDT Home Fibr, Converge ICT, Globe At Home, Smart) and common consumer Wi-Fi routers (Tenda, Huawei, Fiberhome ONTs acting as DNS forwarders) routinely block, sinkhole, or fail to resolve free tunnel top-level domains—especially `*.ngrok-free.dev` and `*.ngrok-free.app`—via ISP threat-filtering feeds because free ngrok domains are heavily abused by phishing sites. When a user on default ISP DNS opens the app:

1. The browser resolves `edselrenz.github.io` without issue (because `github.io` is a trusted global domain).
2. The Angular app immediately calls `GET https://anybody-cube-surprise.ngrok-free.dev/api/v1/health`.
3. The user's router/ISP DNS returns `NXDOMAIN` or a timed-out/sinkholed IP, causing Chrome/Safari/WebView to abort the request with `net::ERR_NAME_NOT_RESOLVED` (HTTP status `0`).
4. `authInterceptor` catches status `0` and flips the app into **Offline Mode**, forcing the user to manually reconfigure their OS or browser Secure DNS (DoH) to Cloudflare (`1.1.1.1`) or Google (`8.8.8.8`) just to resolve the backend hostname.

1.3 Why Problem #2 Happens: Manual Tunnel Entry (and the Broken UI Paradox)

To work around ngrok DNS failures, `scripts/start_server.ps1` starts a second tunnel using Cloudflare Quick Tunnels (`*.trycloudflare.com`). However, this creates three compounding issues:

1. **Random URL on Every Restart:** Because `cloudflared tunnel --url` is an unauthenticated Quick Tunnel, Cloudflare generates a brand-new random 4-word subdomain (`https://word1-word2-word3-word4.trycloudflare.com`) every time `start_server.bat` is restarted. It cannot be hardcoded once at build time.
2. **UI Button Removed While Script Still Instructs Users to Click It:** Notice that `scripts/start_server.ps1` (Lines 170–171) still prints: *"Or in the App: Tap the [Settings] gear on the Offline banner and paste the Cloudflare Tunnel URL ($cfUrl) directly!"* Yet in commit `06cefc5`, the `[Set Server]` / Settings button was removed from `app.component.html`! Users currently have **no UI button** to enter the Cloudflare Tunnel URL even if they wanted to.
3. **Some Strictly Filtered Networks Also Block `*.trycloudflare.com`:** Corporate, campus, and strict ISP filters increasingly categorize ephemeral `*.trycloudflare.com` quick tunnels under security blocklists alongside `*.ngrok-free.dev`.

2. Architectural Alternatives Comparison Matrix

To eliminate both manual DNS reconfiguration and manual tunnel URL entry, we evaluate four concrete engineering solutions across cost, ISP DNS compatibility, setup effort, and laptop dependency:

Architecture Option	Cost	Fixes ISP DNS Block Without User Action?	Eliminates Manual Tunnel URL Entry?	How It Works & Trade-Offs
Option 1 (Recommended for Production): Cloud PaaS Backend (Render / Koyeb / Railway) + Managed Postgres (Neon / Supabase)	**$0 / month** (Free Tiers)	**YES (100%)** Uses standard commercial domains (`*.onrender.com`, `*.koyeb.app`) never blocked by PLDT/Globe/Converge DNS.	**YES (100%)** Permanent static HTTPS URL baked once into `environment.prod.ts`.	Deploys `backend/` + PostgreSQL to the cloud. Works 24/7 even when the developer's laptop is off. Note: Render free tier spins down after 15 min of idle (~25s cold start, solvable with a free cron ping).
Option 2 (Best for Laptop Hosting with Zero Cost): Cloudflare Worker Reverse Proxy (`*.workers.dev`) + Auto-Sync in `start_server.ps1`	**$0 / month** (100,000 free reqs/day)	**YES (~99%)** Browser talks only to fixed `*.workers.dev` edge; Worker proxies inside Cloudflare to ngrok/cloudflared.	**YES (100%)** `start_server.ps1` auto-pushes the new `$cfUrl` to the Worker KV on startup.	Keeps FastAPI & Postgres on the developer's laptop. When `start_server.bat` runs, it automatically registers the new random `$cfUrl` and `$ngrokUrl` with the fixed Cloudflare Worker proxy.
Option 3 (Simplest Zero-Account Code Fix for Laptop Hosting): Auto-Published `runtime-config.json` on GitHub + Multi-Endpoint Auto-Failover	**$0 / month** (Uses existing GitHub repo/Gist)	**PARTIAL (~85%)** Auto-switches to `*.trycloudflare.com` when `*.ngrok-free.dev` fails DNS, plus DoH diagnostic fallback.	**YES (100%)** `start_server.ps1` writes `$cfUrl` to GitHub Gist/raw JSON; Angular fetches it automatically on load.	Requires zero external cloud accounts beyond GitHub. Since `edselrenz.github.io` / `gist.githubusercontent.com` is never blocked by ISP DNS, the app always discovers the latest live tunnel URL automatically.

Architecture Option	Cost	Fixes ISP DNS Block Without User Action?	Eliminates Manual Tunnel URL Entry?	How It Works & Trade-Offs
Option 4 (Permanent Custom Domain Tunnel): Cloudflare Named Tunnel (`cloudflared tunnel create`) on Custom Domain	**~$1.50–$9 / year** (Domain registration only)	**YES (100%)** Uses a dedicated custom domain (e.g. `api.ne-travel.xyz`) on Cloudflare authoritative DNS.	**YES (100%)** Named tunnels have a permanent static hostname that never changes on restart.	Replaces ephemeral Quick Tunnels (`--url`) with an authenticated Named Tunnel (`cloudflared tunnel run ne-travel-api`) bound to a custom CNAME record.

3. Recommended Dual-Track Implementation Plan

To solve your problem immediately while keeping your local laptop workflow working seamlessly—and providing a clean path to 24/7 cloud hosting—we recommend a **two-stage implementation plan**:

- **Stage 1 (Immediate, 15-Minute Local Laptop Upgrade — Zero Cost, Zero Manual URL Entry, Automatic Failover):** Upgrade `scripts/start_server.ps1` and Angular's `ConnectivityService` so that whenever you run `start_server.bat`, the script automatically publishes the newly generated Cloudflare Tunnel URL (`$cfUrl`) and ngrok URL (`$ngrokUrl`) to a fixed **Cloudflare Worker Proxy / GitHub Gist Registry**. The GitHub Pages app automatically fetches the live tunnel URL on startup and races both endpoints so users never touch a tunnel setting or DNS configuration.
- **Stage 2 (Permanent 24/7 Production Upgrade — Cloud PaaS + Managed PostgreSQL):** Deploy the FastAPI backend to **Render** (or **Koyeb**) and the PostgreSQL database to **Neon.tech** (or **Supabase**) using a `render.yaml` blueprint, eliminating local tunnels and host-laptop uptime requirements completely.

4. Stage 1 Blueprint: Automated Tunnel Publishing & Cloudflare Worker Reverse Proxy

4.1 Architecture Data Flow

1. **Developer Runs `start_server.bat` on Laptop:** PowerShell starts PostgreSQL/SQLite, FastAPI (`127.0.0.1:8000`), ngrok (`anybody-cube-surprise.ngrok-free.dev`), and Cloudflare Tunnel (`https://random-words.trycloudflare.com`).
2. **Automatic Endpoint Registration (New Step in `start_server.ps1`):** As soon as `$cfUrl` is parsed from the log, `start_server.ps1` sends an automated `Invoke-RestMethod` request to update:
   - A free **Cloudflare Worker Reverse Proxy** (e.g., `https://ne-itinerary-api.<subdomain>.workers.dev`) and/or
   - A public **GitHub Gist** (`runtime-endpoints.json`) hosted on `gist.githubusercontent.com` (which is never blocked by Philippine ISP DNS).
3. **Zero-Touch Client Connection on GitHub Pages:** When any user opens https://edselrenz.github.io/Itinerary-App/:
   - The app calls the fixed Cloudflare Worker Proxy URL (or reads the live `$cfUrl` from the GitHub Gist registry).
   - If the Worker Proxy is used, the user's browser only ever resolves `*.workers.dev` (bypassing PLDT/Converge/Tenda DNS blocks on `*.ngrok-free.dev`), and the Worker forwards the request over Cloudflare's internal edge directly to your active `$cfUrl` or `$ngrokUrl`!

4.2 Cloudflare Worker Reverse Proxy Script (`cloudflare-worker/worker.js`)

Deploy this single-file Worker on the free Cloudflare dashboard (`Workers & Pages → Create Worker`, bind a KV namespace named `TUNNEL_KV`, and set secret `ADMIN_SYNC_KEY`). It gives your GitHub Pages app a **permanent static HTTPS URL** that proxies to your laptop's latest random Cloudflare Tunnel or ngrok tunnel:

```
export default {
  async fetch(request, env) {
    const url = new URL(request.url);
    const corsHeaders = {
      'Access-Control-Allow-Origin': 'https://edselrenz.github.io',
      'Access-Control-Allow-Methods': 'GET, POST, PATCH, PUT, DELETE, OPTIONS',
      'Access-Control-Allow-Headers': 'Content-Type, Authorization, ngrok-skip-browser-warning',
      'Access-Control-Allow-Credentials': 'true',
    };

    if (request.method === 'OPTIONS') {
      return new Response(null, { status: 204, headers: corsHeaders });
    }

    // 1. Admin endpoint called automatically by scripts/start_server.ps1 on laptop startup
    if (url.pathname === '/__sync_tunnel' && request.method === 'POST') {
      const authKey = request.headers.get('X-Admin-Sync-Key');
      if (!env.ADMIN_SYNC_KEY || authKey !== env.ADMIN_SYNC_KEY) {
        return new Response(JSON.stringify({ error: 'Unauthorized' }), { status: 401, headers:
corsHeaders });
      }
      const body = await request.json();
      await env.TUNNEL_KV.put('ACTIVE_TUNNELS', JSON.stringify({
        cfUrl: body.cfUrl || null,
        ngrokUrl: body.ngrokUrl || 'https://anybody-cube-surprise.ngrok-free.dev',
        updatedAt: new Date().toISOString(),
      }));
      return new Response(JSON.stringify({ status: 'synced' }), { status: 200, headers: corsHeaders
});
    }

    // 2. Proxy all /api/v1/* requests to the active Cloudflare Tunnel first, falling back to ngrok
    const stored = await env.TUNNEL_KV.get('ACTIVE_TUNNELS', { type: 'json' });
    const candidates = [
      stored?.cfUrl,
      stored?.ngrokUrl || 'https://anybody-cube-surprise.ngrok-free.dev',
    ].filter(Boolean);

    let lastError = null;
    for (const base of candidates) {
      try {
        const targetUrl = `${base.replace(/\/+$/, '')}${url.pathname}${url.search}`;
        const proxyHeaders = new Headers(request.headers);
        proxyHeaders.set('ngrok-skip-browser-warning', 'true');
        proxyHeaders.delete('Host');

        const upstreamRes = await fetch(targetUrl, {
          method: request.method,
          headers: proxyHeaders,
          body: ['GET', 'HEAD'].includes(request.method) ? null : request.body,
        });

        if (upstreamRes.status < 502) {
          const resHeaders = new Headers(upstreamRes.headers);
          Object.entries(corsHeaders).forEach(([k, v]) => resHeaders.set(k, v));
          return new Response(upstreamRes.body, { status: upstreamRes.status, headers: resHeaders
});
        }
      } catch (err) {
        lastError = err;
      }
    }

    return new Response(
      JSON.stringify({ detail: 'Host backend offline', error: String(lastError) }),
      { status: 502, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    );
```

```
  },
};
```

4.3 Auto-Sync Addition to `scripts/start_server.ps1` (After Line 152)

Add this block to `scripts/start_server.ps1` right after `$cfUrl` is detected so your laptop automatically publishes its new tunnel URL to the Worker (and/or a GitHub Gist) without any human copy-pasting:

```
# 3c. Automatically publish $cfUrl and $ngrokUrl to Cloudflare Worker Proxy & GitHub Gist
if ($cfUrl -or $ngrokUrl) {
    # Option A: Push to Cloudflare Worker Proxy (if WORKER_SYNC_URL & WORKER_SYNC_KEY are in .env)
    if ($env:WORKER_SYNC_URL -and $env:WORKER_SYNC_KEY) {
        try {
            $syncPayload = @{ cfUrl = $cfUrl; ngrokUrl = $ngrokUrl } | ConvertTo-Json
            Invoke-RestMethod -Uri "$($env:WORKER_SYNC_URL)/__sync_tunnel" `
                -Method Post `
                -Headers @{ "X-Admin-Sync-Key" = $env:WORKER_SYNC_KEY; "Content-Type" =
"application/json" } `
                -Body $syncPayload -TimeoutSec 5 | Out-Null
            Write-Host "  [OK] Auto-synced active tunnel URLs to Cloudflare Worker Proxy!" -
ForegroundColor Green
        } catch {
            Write-Host "  [WARN] Could not auto-sync to Worker Proxy: $_" -ForegroundColor Yellow
        }
    }

    # Option B: Push to GitHub Gist Runtime Discovery Registry (if GITHUB_GIST_ID &
GITHUB_GIST_TOKEN are in .env)
    if ($env:GITHUB_GIST_ID -and $env:GITHUB_GIST_TOKEN) {
        try {
            $runtimeJson = @{
                updated_at = (Get-Date).ToUniversalTime().ToString("o")
                primary_url = if ($cfUrl) { "$cfUrl/api/v1" } else { "$ngrokUrl/api/v1" }
                fallback_urls = @("$cfUrl/api/v1", "$ngrokUrl/api/v1") | Where-Object { $_ -and $_ -
notmatch '^/api' }
            } | ConvertTo-Json -Compress
            $gistBody = @{
                files = @{ "ne_runtime_endpoints.json" = @{ content = $runtimeJson } }
            } | ConvertTo-Json -Depth 4
            Invoke-RestMethod -Uri "https://api.github.com/gists/$($env:GITHUB_GIST_ID)" `
                -Method Patch `
                -Headers @{
                    "Authorization" = "Bearer $($env:GITHUB_GIST_TOKEN)"
                    "Accept" = "application/vnd.github+json"
                    "User-Agent" = "NuevaEcijaItineraryApp-Launcher"
                } -Body $gistBody -TimeoutSec 5 | Out-Null
            Write-Host "  [OK] Auto-published tunnel URL ($cfUrl) to GitHub Gist discovery
registry!" -ForegroundColor Green
        } catch {
            Write-Host "  [WARN] Could not update GitHub Gist registry: $_" -ForegroundColor Yellow
        }
    }
}
```

4.4 Smart Auto-Discovery & Failover in Angular (`ConnectivityService`)

Update `Project/src/app/core/services/connectivity.service.ts` so that on startup—or whenever the default endpoint fails health check—it automatically queries the GitHub Gist discovery JSON (`gist.githubusercontent.com`, which is never blocked by PLDT/Converge DNS) and switches `ne_custom_api_url` to the active Cloudflare Tunnel URL without any user action:

```
// Inside ConnectivityService (Project/src/app/core/services/connectivity.service.ts)
private readonly discoveryRegistryUrl =
  'https://gist.githubusercontent.com/Edselrenz/<GIST_ID>/raw/ne_runtime_endpoints.json';

public checkHostHealth(): Observable<boolean> {
  this.isCheckingHealthSubject.next(true);
  const healthUrl = `${environment.apiBaseUrl}/health`;
```

```
    return this.http.get<HealthResponse>(healthUrl).pipe(
      tap(() => {
        this.isCheckingHealthSubject.next(false);
        this.setHostOffline(false);
      }),
      map(() => true),
      catchError(() => this.tryAutoDiscoverTunnel())
    );
  }

  private tryAutoDiscoverTunnel(): Observable<boolean> {
    return this.http
      .get<{ primary_url?: string; fallback_urls?: string[] }>(
        `${this.discoveryRegistryUrl}?t=${Date.now()}`
      )
      .pipe(
        switchMap((registry) => {
          const candidates = [
            registry?.primary_url,
            ...(registry?.fallback_urls || []),
          ].filter((u): u is string => !!u && this.isValidApiUrl(u));

          if (candidates.length === 0) {
            return of(false);
          }
          // Probe the discovered Cloudflare Tunnel URL first
          const bestUrl = candidates[0];
          this.setCustomApiUrl(bestUrl);
          return this.http.get<HealthResponse>(`${environment.apiBaseUrl}/health`).pipe(
            tap(() => {
              this.isCheckingHealthSubject.next(false);
              this.setHostOffline(false);
            }),
            map(() => true),
            catchError(() => of(false))
          );
        }),
        catchError(() => {
          this.isCheckingHealthSubject.next(false);
          this.isHostOfflineSubject.next(true);
          return of(false);
        })
      );
  }
```

5. Stage 2 Blueprint: Permanent Cloud PaaS Deployment (Eliminating Tunnels Completely)

If you want the application to work 24/7 even when your laptop is powered off—and completely eliminate both `ngrok` and `cloudflared`—deploy the `backend/` folder to **Render.com** (free Web Service tier) connected to a free **Neon.tech** PostgreSQL database (or bundled SQLite for read-only/demo mode).

5.1 Add a Root `render.yaml` Infrastructure-as-Code Blueprint

Adding this file to the root of your GitHub repository allows one-click deployment on Render (`New → Blueprint`):

```
services:
  - type: web
    name: nueva-ecija-itinerary-api
    runtime: python
    rootDir: backend
    plan: free
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
    healthCheckPath: /api/v1/health
```

```
    envVars:
      - key: APP_ENV
        value: pilot
      - key: JWT_SECRET_KEY
        generateValue: true
      - key: DATABASE_URL
        sync: false  # Paste your free Neon.tech or Supabase PostgreSQL URI here
      - key: CORS_ORIGINS
        value:
'["https://edselrenz.github.io","http://localhost:4200","http://localhost:8100","capacitor://localho
st"]'
```

5.2 Update Trusted Hostnames in `environment.prod.ts` & `connectivity.service.ts`

Once deployed to Render (or Cloudflare Workers), add `.onrender.com` and `.workers.dev` to `isTrustedApiHostname()` in `Project/src/environments/environment.prod.ts` and `Project/src/app/core/services/connectivity.service.ts` and set the default `apiBaseUrl` to your permanent cloud domain:

```
  if (
    lower === 'anybody-cube-surprise.ngrok-free.dev' ||
    lower.endsWith('.onrender.com') ||
    lower.endsWith('.koyeb.app') ||
    lower.endsWith('.workers.dev') ||
    lower.endsWith('.ngrok-free.dev') ||
    lower.endsWith('.trycloudflare.com')
  ) {
    return true;
  }
```

6. Summary Decision Guide

- **If you must keep running the backend on your local laptop:** Implement **Stage 1** (auto-publishing `$cfUrl` in `scripts/start_server.ps1` to a GitHub Gist discovery file + `tryAutoDiscoverTunnel()` in `ConnectivityService`, or fronting it with a free `*.workers.dev` Cloudflare Worker proxy). Users will never need to enter a tunnel URL or change their DNS again.
- **If you want a true production deployment that works when your laptop is off:** Implement **Stage 2** (`render.yaml` on Render/Koyeb + Neon PostgreSQL). This replaces both `ngrok` and `cloudflared` with a standard commercial HTTPS endpoint (`*.onrender.com`) that works across every ISP in the Philippines out of the box.